

Análisis del problema: Agencia de Detectives – Caso Misterioso

1. Descripción general

La agencia de detectives necesita un sistema para administrar la información de un caso de investigación.

Cada caso debe guardar:

- La información general del caso.
- Un máximo de cinco ubicaciones de investigación.
- Una cantidad variable de pistas.
- Información estadística sobre las ubicaciones y las pistas.

Las ubicaciones se almacenarán en un arreglo básico de tamaño fijo porque el caso solamente puede tener cinco posiciones disponibles. Las pistas se almacenarán en un `ArrayList<Pista>` porque durante la investigación no se conoce cuántas pistas serán encontradas.

El sistema debe permitir registrar, consultar, buscar, modificar y eliminar información. También deberá validar los datos y manejar excepciones para evitar que el programa finalice inesperadamente.

Requisitos funcionales

- 1.
2. El programa debe solicitar el nombre, código y detective responsable del caso.
3. Debe existir un caso activo durante la ejecución del programa.
4. El usuario podrá reemplazar el caso activo por uno nuevo.
5. Cada caso tendrá un arreglo básico con cinco posiciones para almacenar ubicaciones.
6. Las posiciones que no contengan una ubicación tendrán el valor null.
7. Cada ubicación tendrá código, nombre, dirección, nivel de riesgo y estado.
8. El nivel de riesgo debe estar entre 1 y 10.
9. El usuario debe seleccionar la posición donde desea registrar una ubicación.
10. La posición seleccionada debe encontrarse entre 0 y 4.
11. No se podrá registrar una ubicación en una posición ocupada.

- 12.El sistema permitirá consultar todas las ubicaciones registradas.
- 13.El sistema permitirá consultar una ubicación específica mediante su posición.
- 14.No se deberán utilizar los métodos de una posición que contenga null.
- 15.El sistema permitirá modificar el nivel de riesgo y el estado de una ubicación.
- 16.El sistema permitirá descartar una ubicación, colocando nuevamente null en su posición.
- 17.Las pistas se almacenarán en un ArrayList<Pista>.
- 18.Cada pista tendrá código, descripción, tipo de evidencia, importancia y confiabilidad.
- 19.La importancia de una pista debe estar entre 1 y 10.
- 20.La confiabilidad de una pista debe estar entre 0 y 100.
- 21.No se podrán registrar dos pistas con el mismo código.
- 22.El sistema permitirá consultar todas las pistas registradas.
- 23.El sistema permitirá buscar una pista mediante su código.
- 24.El sistema permitirá modificar una pista previamente registrada.
- 25.El sistema permitirá eliminar una pista del ArrayList.
- 26.El sistema calculará la cantidad de ubicaciones registradas.
- 27.El sistema calculará la cantidad de espacios disponibles.
- 28.El sistema determinará la ubicación con mayor nivel de riesgo.
- 29.El sistema calculará la cantidad de pistas registradas.
- 30.El sistema determinará la pista con mayor importancia.
- 31.El sistema determinará la pista con mayor confiabilidad.
- 32.El sistema calculará el promedio de importancia de las pistas.
- 33.Los cálculos relacionados con pistas no deben realizarse si el ArrayList está vacío.
- 34.Las entradas numéricas incorrectas se manejarán con InputMismatchException.
- 35.Los valores fuera de los rangos permitidos producirán una IllegalArgumentException.

36. Las posiciones inexistentes serán controladas mediante `IndexOutOfBoundsException`.
37. Las posiciones ocupadas o vacías se controlarán mediante `IllegalStateException`.
38. El programa utilizará bloques try-catch.
39. El programa utilizará al menos un bloque finally.
40. Después de una excepción controlada, el programa deberá continuar funcionando.
41. El programa presentará un menú de 13 opciones.

Clases identificadas

La solución utiliza las siguientes clases:

1. Ubicacion
2. Pista
3. CasoInvestigacion
4. Main

Clase Ubicacion

Propósito

La clase Ubicacion representa un lugar relacionado con la investigación. Cada objeto almacena los datos de una ubicación específica.

Esta clase también es responsable de validar que el nivel de riesgo esté entre 1 y 10.

Atributo	Tipo	Visibilidad	Descripción
codigo	String	private	Identificador de la ubicación.
nombre	String	private	Nombre del lugar investigado.

direccion	String	private	Dirección o descripción del lugar.
nivelRiesgo	int	private	Riesgo de la ubicación entre 1 y 10.
estado	String	private	Estado actual de la ubicación.

Método	Retorno	Descripción
getCodigo()	String	Devuelve el código.
getNombre()	String	Devuelve el nombre.
getDireccion()	String	Devuelve la dirección.
getNivelRiesgo()	int	Devuelve el nivel de riesgo.
getEstado()	String	Devuelve el estado.
setCodigo(String codigo)	void	Asigna un código válido.
setNombre(String nombre)	void	Asigna un nombre válido.
setDireccion(String direccion)	void	Asigna una dirección válida.
setNivelRiesgo(int nivelRiesgo)	void	Valida y asigna un riesgo entre 1 y 10.
setEstado(String estado)	void	Asigna un estado válido.
validarTexto(String valor, String campo)	void	Comprueba que un texto no sea null ni esté vacío.
toString()	String	Devuelve los datos de la ubicación como texto.

Clase Pista

Propósito

La clase Pista representa una evidencia encontrada durante la investigación.

Es responsable de almacenar la información de una pista y validar su importancia y confiabilidad.

Atributo	Tipo	Visibilidad	Descripción
codigo	String	private	Identificador único de la pista.
descripcion	String	private	Explicación de la pista encontrada.
tipoEvidencia	String	private	Clasificación de la evidencia.
nivelImportancia	int	private	Importancia entre 1 y 10.
nivelConfiabilidad	int	private	Confiabilidad entre 0 y 100.

Método	Retorno	Descripción
getCodigo()	String	Devuelve el código de la pista.
getDescripcion()	String	Devuelve la descripción.
getTipoEvidencia()	String	Devuelve el tipo de evidencia.
getNivelImportancia()	int	Devuelve la importancia.
getNivelConfiabilidad()	int	Devuelve la confiabilidad.
setCodigo(String codigo)	void	Asigna un código válido.
setDescripcion(String descripcion)	void	Asigna una descripción válida.
setTipoEvidencia(String tipo)	void	Asigna el tipo de evidencia.
setNivelImportancia(int nivel)	void	Valida un nivel entre 1 y 10.
setNivelConfiabilidad(int nivel)	void	Valida un nivel entre 0 y 100.
validarTexto(String valor, String campo)	void	Valida que un texto no esté vacío.
toString()	String	Devuelve los datos de la pista como texto.

Métodos de ubicaciones

Método	Retorno	Descripción
--------	---------	-------------

registrarUbicacion(int posicion, Ubicacion ubicacion)	void	Guarda una ubicación en una posición disponible.
obtenerUbicacion(int posicion)	Ubicacion	Devuelve la ubicación almacenada en una posición.
consultarUbicaciones()	String	Recorre el arreglo y muestra las posiciones que no son null.
modificarUbicacion(int posicion, int riesgo, String estado)	void	Modifica el riesgo y estado de una ubicación.
descartarUbicacion(int posicion)	void	Coloca null en la posición seleccionada.
validarPosicion(int posicion)	void	Verifica que la posición esté entre 0 y 4.
contarUbicaciones()	int	Cuenta las posiciones diferentes de null.
contarEspaciosDisponibles()	int	Calcula los espacios disponibles.
ubicacionMayorRiesgo()	Ubicacion	Devuelve la ubicación con mayor riesgo.

Métodos de pistas

Método	Retorno	Descripción
registrarPista(Pista pista)	void	Agrega una pista si su código no está repetido.
buscarPista(String codigo)	Pista	Recorre el ArrayList y devuelve la pista encontrada.
consultarPistas()	String	Devuelve todas las pistas registradas.
modificarPista(...)	void	Modifica los datos de una pista existente.
eliminarPista(String codigo)	boolean	Elimina una pista y devuelve si fue encontrada.
contarPistas()	int	Devuelve el tamaño del ArrayList.
pistaMayorImportancia()	Pista	Devuelve la pista con mayor importancia.
pistaMayorConfiabilidad()	Pista	Devuelve la pista con mayor confiabilidad.
promedioImportancia()	double	Calcula el promedio de importancia.
generarReporte()	String	Reúne los resultados de la investigación.

Clase Main

Propósito

La clase Main es el driver program solicitado. Contiene el método main(), muestra el menú y controla la interacción entre el usuario y los objetos.

También se encarga de leer los datos con Scanner y manejar las excepciones producidas durante la ejecución.

Atributo	Tipo	Visibilidad	Descripción
TECLADO	Scanner	private static final	Permite leer los datos ingresados.
casoActual	CasoInvestigacion	private static	Mantiene la instancia del caso activo.

Método	Retorno	Descripción
main(String[] args)	void	Inicia y controla la ejecución.
mostrarMenu()	void	Muestra las 13 opciones.
ejecutarOpcion(int opcion)	void	Ejecuta la opción seleccionada.
leerEntero(String mensaje)	int	Lee un entero y maneja InputMismatchException.
leerTexto(String mensaje)	String	Lee un texto y evita cadenas vacías.
crearNuevoCaso()	void	Crea o reemplaza el caso actual.
registrarUbicacion()	void	Solicita y registra una ubicación.
consultarUnaUbicacion()	void	Solicita una posición y muestra su ubicación.
modificarUbicacion()	void	Solicita los nuevos datos de la ubicación.
descartarUbicacion()	void	Elimina una ubicación del arreglo.
registrarPista()	void	Solicita y registra una pista.
leerDatosPista()	Pista	Solicita los datos y construye una pista.
buscarPista()	void	Busca y muestra una pista.
modificarPista()	void	Solicita y modifica los datos de una pista.
eliminarPista()	void	Solicita un código y elimina la pista.

Preguntas

1. ¿Qué propiedades y métodos tendrá cada clase?

Se identificaron cuatro clases:

- Ubicacion: contiene los datos y operaciones de una ubicación.
- Pista: contiene los datos y operaciones de una pista.
- CasoInvestigacion: contiene los datos generales, el arreglo de ubicaciones, el ArrayList de pistas y las operaciones administrativas.
- Main: contiene el menú, las entradas del usuario y el control del programa.

Las propiedades y métodos concretos son los indicados en las tablas anteriores.

2. ¿Qué tipo deben tener las propiedades y métodos?

Los datos textuales utilizan String. Los niveles de riesgo, importancia y confiabilidad utilizan int. El promedio utiliza double.

Las ubicaciones se almacenan en Ubicacion[] y las pistas en ArrayList<Pista>.

Los métodos que solamente realizan una acción utilizan void. Los métodos de búsqueda devuelven un objeto. Los métodos de conteo devuelven int y el promedio devuelve double.

3. ¿Qué propiedad utilizará un arreglo básico?

La propiedad será:

```
private Ubicacion[] ubicaciones;
```

Almacenará objetos de tipo Ubicacion y tendrá un tamaño fijo de cinco posiciones.

```
ubicaciones = new Ubicacion[5];
```

4. ¿Qué propiedad utilizará un ArrayList?

La propiedad será:

```
private ArrayList<Pista> pistas;
```

Almacenará objetos de tipo Pista.

Se utiliza porque no se conoce anticipadamente la cantidad de pistas que se encontrarán.

5. ¿Cuáles serán los modificadores de visibilidad?

Los atributos serán private para cumplir con el encapsulamiento.

Los constructores, getters, setters y operaciones que otras clases necesitan utilizar serán public.

Los métodos auxiliares que solamente se utilizan dentro de una misma clase serán private.

Los métodos y atributos utilizados directamente por main() se declaran como static.

6. ¿Qué parámetros requieren los métodos?

Los métodos reciben solamente los datos necesarios para realizar cada operación. Por ejemplo:

```
registrarUbicacion(int posicion, Ubicacion ubicacion)
```

Necesita la posición y la ubicación.

buscarPista(String codigo)

Necesita el código de la pista.

```
modificarUbicacion(  
    int posicion,  
    int nuevoRiesgo,  
    String nuevoEstado  
)
```

Necesita la posición y los nuevos datos.

```
modificarPista(  
    String codigo,  
    String descripcion,  
    String tipoEvidencia,  
    int importancia,  
    int confiabilidad  
)
```

Necesita el código para localizar la pista y los valores que serán modificados.

7. ¿Cómo se proporcionarán los valores iniciales?

Los valores iniciales se proporcionarán mediante los constructores.

Al crear un caso se inicializará automáticamente:

```
ubicaciones = new Ubicacion[5];
```

```
pistas = new ArrayList<Pista>();
```

Antes de modificar los objetos se deberá validar:

- Que los textos no sean null o estén vacíos.
- Que el riesgo esté entre 1 y 10.
- Que la importancia esté entre 1 y 10.
- Que la confiabilidad esté entre 0 y 100.

- Que la posición se encuentre entre 0 y 4.
- Que la posición del arreglo tenga o no una ubicación, según la operación.
- Que el código de una nueva pista no esté repetido.

8. ¿Cómo se determina si una posición contiene una ubicación?

La posición se compara con null.

```
if (ubicaciones[posicion] == null) {  
    System.out.println("La posición está vacía.");  
}
```

Para recorrer únicamente las ubicaciones existentes:

```
for (int i = 0; i < ubicaciones.length; i++) {  
    if (ubicaciones[i] != null) {  
        System.out.println(ubicaciones[i]);  
    }  
}
```

La comparación debe realizarse antes de llamar métodos sobre el objeto para evitar un NullPointerException.

9. ¿Cómo se buscará, modificará y eliminará en el ArrayList?

Búsqueda

Se recorrerá el ArrayList y se comparará el código:

```
for (Pista pista : pistas) {  
    if (pista.getCodigo().equalsIgnoreCase(codigoBuscado)) {  
        return pista;  
    }  
}
```

Modificación

Primero se buscará la pista. Si existe, se usarán sus métodos set.

```
Pista pista = buscarPista(codigo);
```

```

if (pista != null) {
    pista.setDescripcion(nuevaDescripcion);
}

```

Eliminación

Primero se localizará la pista y después se eliminará:

```
Pista pista = buscarPista(codigo);
```

```

if (pista != null) {
    pistas.remove(pista);
}

```

No se elimina directamente dentro de un for-each, evitando una posible `ConcurrentModificationException`.

10. ¿Qué situaciones pueden producir excepciones?

Situación	Excepción	Lugar donde se maneja
El usuario escribe texto cuando se espera un número	<code>InputMismatchException</code>	Método <code>leerEntero()</code>
Riesgo menor que 1 o mayor que 10	<code>IllegalArgumentException</code>	Clase <code>Ubicacion</code>
Importancia fuera del rango 1–10	<code>IllegalArgumentException</code>	Clase <code>Pista</code>
Confiabilidad fuera del rango 0–100	<code>IllegalArgumentException</code>	Clase <code>Pista</code>
Texto vacío o null	<code>IllegalArgumentException</code>	Constructores y setters
Código de pista repetido	<code>IllegalArgumentException</code>	<code>registrarPista()</code>

Posición menor que 0 o mayor que 4	IndexOutOfBoundsException	Operaciones del arreglo
Registrar en una posición ocupada	IllegalStateException	registrarUbicacion()
Consultar o modificar una posición vacía	IllegalStateException	obtenerUbicacion()
Realizar cálculos con pistas vacías	Se evita mediante isEmpty()	Métodos de cálculo

El bloque finally se utiliza al registrar una ubicación:

```
try {
    // Registro de la ubicación
} catch (Exception e) {
    // Manejo del error
} finally {
    System.out.println("Operación de registro finalizada.");
}
```

El mensaje del finally se ejecutará tanto si la ubicación fue registrada como si ocurrió una excepción

